

AQ-Mario: a world model for Super Mario Bros 1-1

Implementing LeJEPa's SIGReg for action-conditioned video — the calibration it needs, the instrumentation that catches a bad run, and everything measured along the way

Complete project record · 8 September 2026 · every number measured, none estimated

What this is. A 9.8M-parameter JEPa world model trained on 8,000 episodes of Super Mario Bros 1-1, built as a study of two questions: *does SIGReg transfer out of image self-supervised learning into a world model for control*, and *can a representation's fitness be gated automatically, per checkpoint, instead of discovered after a planner fails*.

What was found. The objective transfers but needs four corrections that are not visible from the image-domain framing: a closed-form null without which λ is not comparable across batch sizes; a collapse-detection power that scales with batch size and therefore conflicts with the memory cost of video; a $25\times$ signal reduction from the BatchNorm the architecture requires; and a broken iid assumption on temporally correlated batches. In this setting the stop-gradient was still required. And at the application level, preventing collapse did not preserve usable state: an uncollapsed representation scored $R^2 = -0.383$ on the single variable a planner needs.

Status. All four stages are implemented and tested (161 automated checks). Results below come from a 4,000-step run — roughly 4% of the planned schedule. The full-budget run and Stages 3–4 are built but not yet executed.

CONTENTS

- | | |
|---|---|
| 1 · The problem and the design | 9 · The collapse, and how it was found |
| 2 · Stage 1 — data and ground truth | 10 · Results |
| 3 · The model | 11 · Anti-collapse $\neq$ usable state |
| 4 · The calibration SIGReg needs | 12 · Instrumentation that pays for itself |
| 5 · Batch size is a SIGReg hyperparameter | 13 · Six bugs, and how each surfaced |
| 6 · BatchNorm halves the job | 14 · Built but not yet run |
| 7 · Correlated batches break the null | 15 · Limitations |
| 8 · The stop-gradient was still required | 16 · Practice, and where everything lives |

1 · The problem and the design

A **world model** learns an environment's dynamics well enough to roll them forward internally: given a few frames and a sequence of actions, predict what follows. This one predicts in a **192-dimensional latent space** rather than in pixels — predicting pixels spends most of the model's capacity on clouds and brick texture, while the latent is meant to hold only what matters for control.

That creates the central difficulty. A latent-space prediction objective has a trivial solution: map every possible frame to the *same* latent, and prediction becomes free. Preventing that collapse is what SIGReg is for, and testing whether it succeeds outside image SSL is what this project is.

The second design commitment: **the run is graded on its representation, not its loss**. Every collapsed model in this project reported an excellent loss. The evaluation is instead a set of frozen-encoder probes — can the height of the character, its position, the camera, and imminent death be read back out of the 192 numbers — run automatically on every checkpoint, with a fail-closed abort.

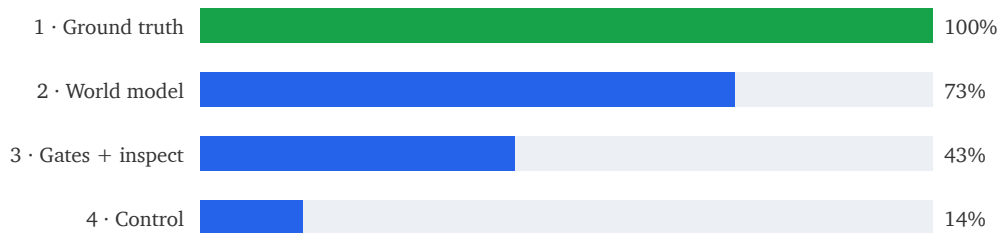


Figure 1. Stage completion as reported by `aqmario.tracker`, which probes real state — files, shard counts, metrics, gate JSON — rather than checkboxes.

2 · Stage 1 — data and ground truth

8,000 episodes across a fleet of 16 machines: **4.88M observations, 27.4 GB compressed** (992 GB of raw pixels; NES artwork deflates 27×).

POLICY	EPISODES	OBSERVATIONS	WHY IT EXISTS	REACHES
jump-biased random	4,000	2,674,774	falls in every pit, hits every enemy — teaches what death looks like	~45%
PPO explorer	4,000	2,202,594	covers the back half; a representation graded only on a level's opening is graded on the easy half	100%

The PPO agent was **stopped early on purpose**, the moment its rolling mean reached 2,400 of the flagpole's 3,161 — an agent that finishes perfectly supplies 4,000 copies of one trajectory, which is worse for coverage than one that dies in interesting places. It peaked at 3,161 (the flagpole) with a mean of 2,424.

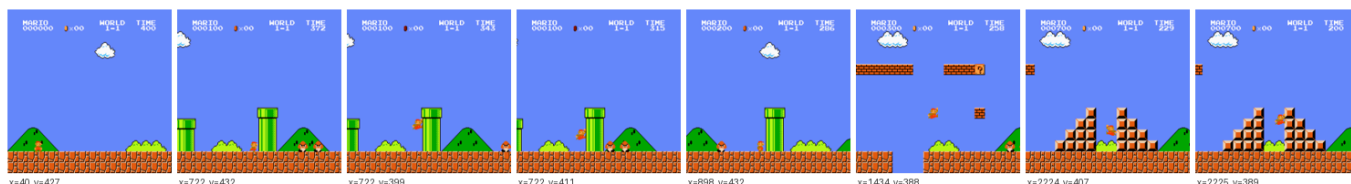


Figure 2. One training episode. `x` and `y` are read from console RAM and are never model inputs — they are the probe targets used to evaluate the frozen representation afterwards.

GROUND TRUTH YOU CAN TRUST

RAM addresses ship as *candidates*, not constants, because a wrong address does not crash — it silently corrupts every probe, gate and planner cost. `scripts/validate_ram.py` picks the winner empirically and `collect_data.py` refuses to run until it has. The chosen player-y pair reproduces the environment's own reported value with $R^2 = 1.0000000000$ over 300 frames, and the production container was verified independently: our RAM read returned `world_x = 87` against the emulator's `info_x = 87`.

3 · The model

COMPONENT	PARAMS	NOTE
encoder — ViT-Tiny patch-14 @ 224px	5.65M	257 tokens/frame → CLS → MLP → BatchNorm(affine=False)
predictor — 6 × AdaLN-Zero, causal	4.04M	3 context frames → next 5 latents
action encoder	0.04M	2 frames × 6 buttons → 192
auxiliary heads	0.07M	height, camera, death-within-5; ablated in the control
total	9.80M	

Four choices that are load-bearing, each of which has a plausible alternative that quietly breaks the run:

- **BatchNorm, not LayerNorm, at the end of the projection.** A trailing LayerNorm puts every embedding on a fixed-radius sphere, which fights the isotropic-Gaussian target directly.
- **Actions enter by AdaLN-Zero, not concatenation.** Concatenated action tokens let the predictor ignore the action for thousands of steps and learn "the next frame resembles this one" — the degenerate solution that leaves height unencoded. AdaLN-Zero starts as an exact identity, so the action is the only thing that can modulate it.
- **Frame skip 2, not 5.** A jump arc is ~30 emulator frames; at skip-5 it is six samples, and vertical information barely helps one-step prediction, so the objective discards it.
- **Future slots are mask tokens under a causal mask.** Slot $t+1$ attends only to slot t , itself a prediction — so multi-step rollout is the training objective rather than something bolted on at evaluation.

The parameter count is 9.8M against the ~15M the original plan assumed. That is reported as under-band rather than the band being widened to fit; closing the gap means raising `predictor_layers`, which is a deliberate experiment, not a rounding.

4 · The calibration SIGReg needs and does not ship with

SIGReg compares the empirical characteristic function of each 1-D random projection to the Gaussian one. Under the null, $E|\varphi_n - \varphi|^2 = (1 - e^{-t^2})/n$ exactly, so integrating against the Gauss–Hermite weight gives a closed form for the value a *perfectly* Gaussian embedding produces:

$$E[T \mid z \sim N(0, I)] = \sqrt{\pi} \cdot (1 - 1/\sqrt{2}) / n = \mathbf{0.51914 / n}$$

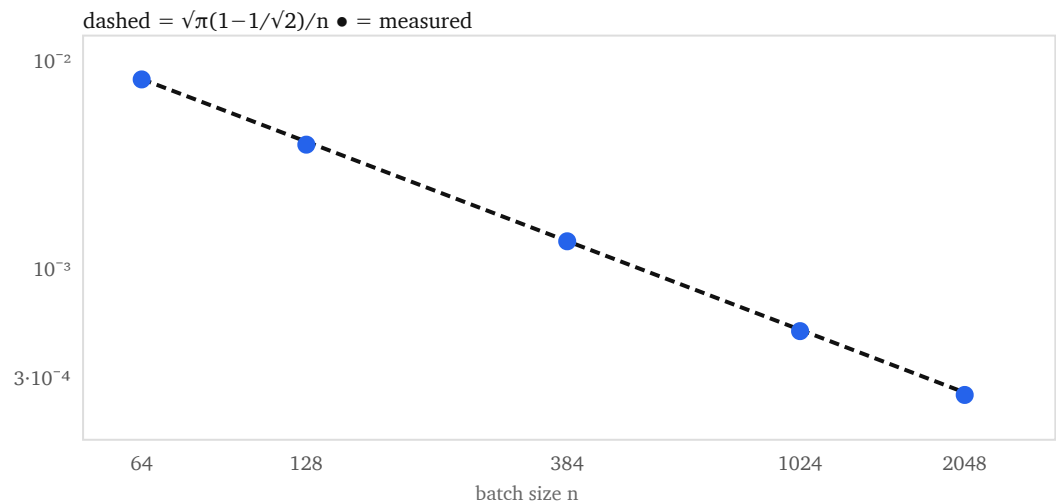


Figure 3. Predicted vs measured null, $n = 64 \dots 2048$; agreement within 3% throughout.

- At batch 64 the term **floors at 0.0013** however Gaussian the encoder becomes. Driving SIGReg toward zero is not the goal; reaching $\approx 1.0\times$ null is.
- Raw values from runs at different batch sizes are **not comparable**, so neither are their λ . The comparable quantity is `ratio = T / null(n)`.
- Since the floor is sampling noise, the optimiser can push slightly *below* $1.0\times$ by making the batch mildly repulsive. Benign, and a second reason the target is a ratio near one.

What the statistic can and cannot see

LATENT PATHOLOGY (N=384, D=192, NULL = 0.00123)	$\times$ NULL
all latents identical	410×
wrong global scale ($z \times 3$)	321×
rank-8 collapse	254×
heavy radial tail	30×
anisotropic covariance	10×
<i>blind spots — the projection CLT hides these:</i>	
5% of points at 8σ	2×
per-coordinate bimodality	1.1×
unit shell at radius $\sqrt{D}$	1.1×

A random 1-D projection of a 192-dim distribution is nearly Gaussian unless the covariance or the radial law is wrong. SIGReg is therefore a guarantee against *collapse* — always a rank or scale failure — not a certificate that the embedding is Gaussian in every respect.

5 · Batch size is a SIGReg hyperparameter



Figure 4. SIGReg in units of its own null for a rank-*r* latent after BatchNorm.

At $n = 32$ a fully collapsed latent reads $2.8\times$ against a healthy $0.4\times$ — barely separable, so the term cannot resist what it can hardly see. At $n = 512$ the same collapse reads $41.8\times$ against $1.0\times$.

THE TENSION SPECIFIC TO VIDEO

A world-model step encodes `batch × window` images — at window 8 and batch 96 that is 768 images and **197,376 ViT tokens**, LLM-scale token throughput from a 9.8M-parameter model. Memory therefore pushes the batch *down*, into exactly the regime where SIGReg is weakest. Activation checkpointing is not a throughput optimisation here; it is what buys the regulariser its statistical power.

6 · BatchNorm halves the job

The projection head must end in BatchNorm (§3). But BatchNorm pins the covariance *diagonal* for free, which is a large part of what SIGReg was detecting: a rank-8 latent reads **254× null raw and 10× after BN**. Still unambiguous, 25× less signal. The two are complementary rather than reinforcing, and SIGReg values from a BN-projected encoder are a different quantity from those without one.

The corresponding positive result, and the single experiment that says the design is sound: SIGReg escapes a rank-8 collapse **only when the gradient flows through the BatchNorm**, as it does in a real encoder. Optimising the latent as a free tensor barely moves it.

STEP	SIGREG	× NULL	EFFECTIVE DIM
0	0.0139	10.0	7.9
100	0.0051	3.8	21.5
300	0.0021	1.6	52.0
600	0.0015	1.1	69.5

7 · Correlated batches break the null

A world-model batch is **B windows × W near-identical frames** — two emulator frames apart at skip-2 — so it holds B independent samples, not B×W. Measured on a synthetic *full-rank* latent batched as 32×6:

HOW IT IS SCORED	READS	VERDICT
flattened, against the n=192 null	6.7×	"collapsing"
flattened, against the honest n=32 null	1.2×	fine
per frame position, averaged	1.1×	fine

Flattening also makes SIGReg penalise the within-window similarity the predictor depends on — it pushes consecutive frames apart while the prediction loss pulls them together. Scoring per frame position keeps every latent, makes the null exact at n = B, and removes the conflict.

The same correction applies to the effective-dimension diagnostic. Marchenko–Pastur gives $E[PR] = nD/(n+D)$, so a genuinely full-rank 192-dim latent reads **128, not 192**, at n = 384 — measured 127.6 against the formula's 128.0, and 175.5 against 175.5 at n = 2048. Inverting it recovers 192.0 ± 1 at every n and stays at 7.6 on a rank-8 latent.

8 · The stop-gradient was still required

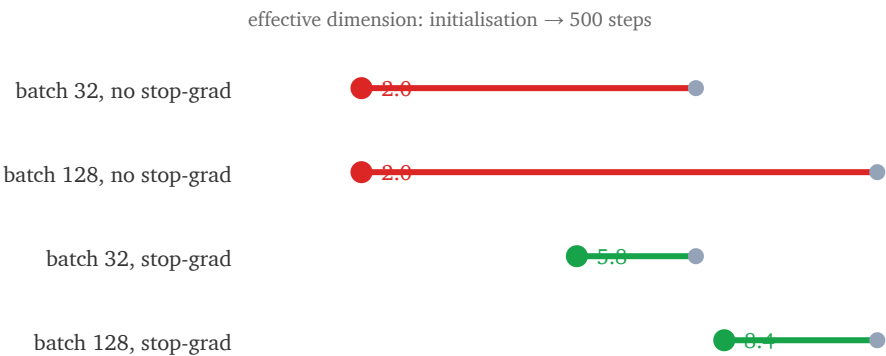


Figure 5. Effective dimension, initialisation → 500 steps. Without the stop-gradient both batch sizes collapse to rank 2 while reporting a 5-step gain of 0.97.

λ alone could not substitute: swept without the stop-gradient it gave effective dimensions of 1.08, 1.89 and 3.50 at $\lambda = 0.1, 1$ and 5 — all collapsed. The arithmetic says why:

λ	PREDICTION LOSS SAVED BY COLLAPSING	SIGREG COST	AUX COST	OUTGUNNED BY
0.1	0.796	0.010	0.008	45×
1.0	0.778	0.065	0.008	11×
5.0	0.745	0.181	0.008	4×

WHY A WORLD MODEL IS A HARDER CASE THAN IMAGE SSL

With a shared encoder, $\text{MSE}(\hat{z}, z_{\text{target}})$ is *directly* minimised by making the target constant, because the target is produced by the same trainable network. Collapse is not a failure mode to be avoided — it is the objective's global optimum unless something removes the incentive. And the prediction task here is unusually easy to trivialise: a rank-1 latent still has unit variance per coordinate after BatchNorm, so the predictor need only predict one scalar.

With the stop-gradient in place λ becomes an effective lever. At $\lambda=10$ the effective dimension *doubles* over training rather than falling:

λ	AUX SCALE	EFFECTIVE DIM (INIT → 1500 STEPS)	GAIN ₅	SIGREG
10	20	10.3 → 21.1	0.654	1.4×
10	1	10.3 → 20.4	0.545	1.5×
1	20	10.3 → 9.7	0.778	3.0×
1	1	10.3 → 8.9	0.817	3.3×

Note the trade-off: $\lambda=1$ predicts better on a narrower latent. Which is *right* is decided by the probe, not by either column — the entire argument for gating on the representation.

9 · The collapse, and how it was found

The first full dry run **passed its gate while collapsing**, and unpicking that produced most of what is now known about this model.

THE TRAP, IN ONE LINE

`pred_loss 0.975 → 0.061`, `gains = 0.939` — an excellent-looking result — with **effective dimension at 1.17 out of a healthy 27**. Every frame of the game had been crushed onto a single direction. The prediction task was trivially solved because there was nothing left to predict, and the gate at the time only asked "did the losses move?", which a collapsing run satisfies perfectly.

A rank-1 latent still has unit variance per coordinate after BatchNorm, so `gain` — the metric intended to be comparable across models — reads near 1.0 on a model that knows nothing. **Effective dimension is the only metric here a collapsed-but-confident run cannot fake**, and it is now the one wired to the abort.

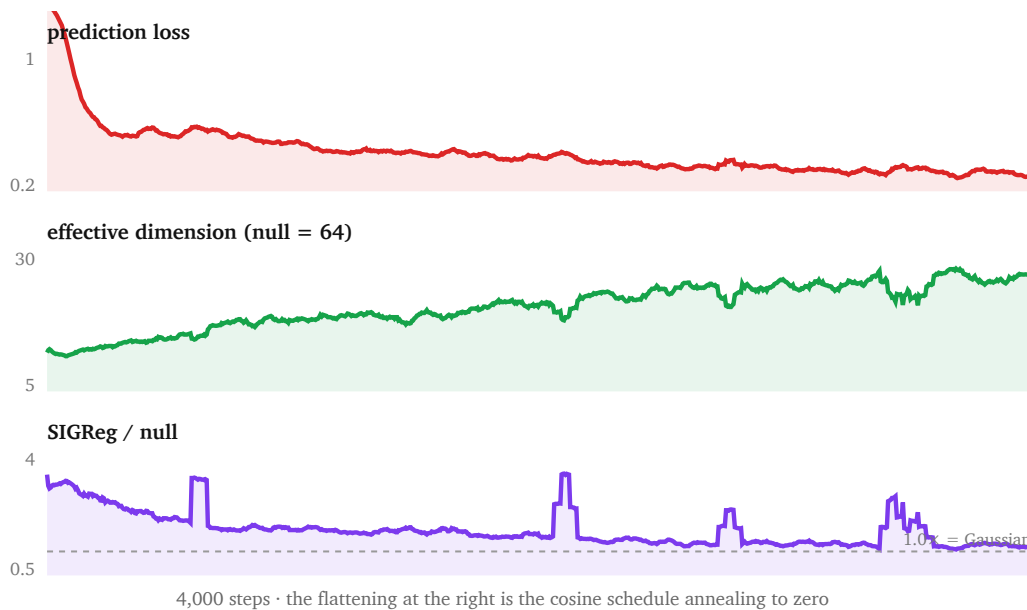


Figure 6. The healthy pattern, from the run all results below come from. Prediction loss falls *while* effective dimension rises. The failure looked identical on the top panel and went to 1 on the middle one.

10 · Results

All from a 4,000-step run — ≈ 0.55 of one epoch, $\sim 4\%$ of the planned schedule — with $\lambda=10$, `aux_scale=20`, stop-gradient on, batch 96. Encoder frozen; probes fit and scored on disjoint sets of complete trajectories.

PROBE	IN-DISTRIBUTION	FULL LEVEL	THRESHOLD
height (y)	0.943	0.796	0.80
position (x)	0.963	0.870	0.95
camera (scroll)	0.962	0.878	0.90
death within 5 steps (AUC)	0.967	0.927	—

Average precision on death is 0.256 against a base rate of 0.0096 — $27\times$ — which is the number to read at a 1% positive rate; accuracy is meaningless there and is deliberately not reported.

The gap between the two columns is the result, not an inconvenience. This model trained on random-play data only — the opening $\sim 45\%$ of the level. "In-distribution" scores it on that same behaviour; "full level" scores it across all 8,000 episodes including the back half it has never seen. Reporting only the first number would be reporting the easy half.

Independent replication and controls

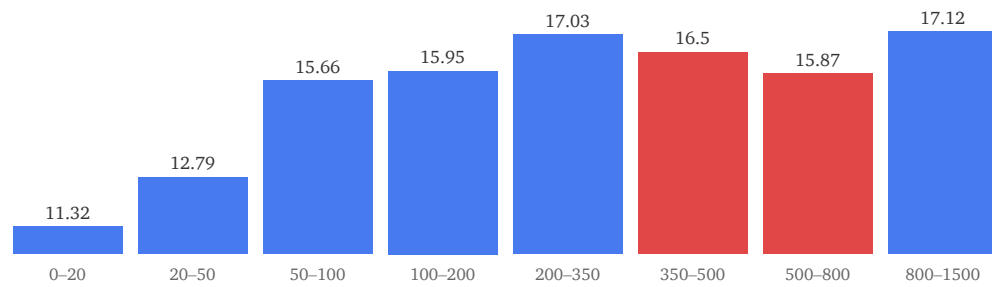
Re-fit and re-scored on shards collected in a separate run, never seen by training:

PREDICTOR OF HEIGHT	R ²
trained encoder (192 dims)	0.932
trained encoder, <i>linear</i> probe only	0.868
raw pixels, 112×112 (12,544 dims)	0.028
raw pixels, 56×56	0.014
random projection of pixels → 192	−0.180
horizontal position alone → height	−0.024
predict the training mean	−0.016
untrained encoder	0.013
trained encoder, <i>shuffled labels</i> (code sanity check)	−0.439

No pixel baseline clears 0.03 at any resolution, so the task is not trivially solvable. The linear probe at 0.868 shows the information is nearly linearly present rather than manufactured by the probe's MLP. Per-episode R² is tight (min 0.896, median 0.931). Held-out frames sit $1.29\times$ further from training frames than training frames sit from each other, so this is not duplicate leakage.

The camera

mean latent distance vs level separation (px)



risers to ~350 px, then dips (red) — the 256 px console screen. Sub-goals are capped at 200 px.

Figure 7. Latent distance against level separation. It rises cleanly to ~350 px and then dips — the console screen is 256 px wide, so two frames about two screens apart start to look alike.

A strict margin test on this fails, and unrolling it is more useful than the number. "Near" means near in *x only* — a character standing, mid-jump and dying all sit at one *x* — so near-pair distance has a floor that has nothing to do with the camera. Decisively, position is recoverable at $R^2 = 0.870$, and a latent you can read position out of is not aliased in the sense the margin claims. The criterion that matters for planning is the range over which distance stays monotone: **350 px measured**, against a planner sub-goal span of 200 px chosen on intuition before any of this existed. The guess and the measurement agree, and the design is now safe for a stated reason.

11 · Anti-collapse ≠ usable state

The control: identical recipe, identical data, identical step count, auxiliary terms removed from the loss entirely. It **did not collapse**, and still could not report the height of the character.

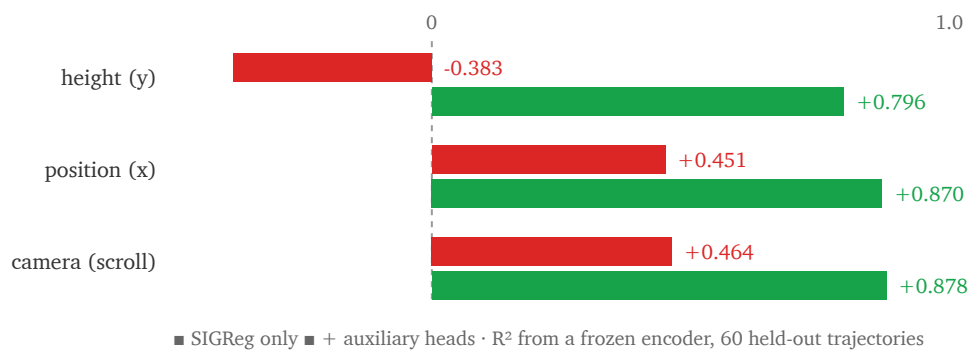


Figure 8. Frozen-encoder probe R², 60 held-out trajectories, full dataset.

PROBE	SIGREG ONLY	+ AUX HEADS (0.07M PARAMS)
height (y)	−0.383	0.796
position (x)	0.451	0.870
camera (scroll)	0.464	0.878
death within 5 (AUC)	0.628	0.927
latent distance usable to	0 px	350 px

An isotropic-Gaussian embedding is a **floor, not a sufficient condition**: it guarantees the representation is non-degenerate, not that it contains what the downstream task requires. A head worth 0.7% of the model's parameters recovers it.

WHAT THIS COMPARISON DOES AND DOES NOT SHOW

The auxiliary variant is *trained* to make height decodable and then *measured* for whether height is decodable. It is not a fair test of representation quality, and it is not evidence that the self-supervised objective learned anything better. What it shows is narrower and still useful: a cheap supervised head fixes the problem, and the probe gate detects its absence at 4% of the training budget with no planner built.

12 · Instrumentation that pays for itself

Every checkpoint — one per 10% of training — saves weights, re-estimates BatchNorm statistics, fits the probes on held-out trajectories, and compares against the previous checkpoint on four axes:

CHECK	QUESTION	THE FAILURE IT MAPS TO
learning	is prediction loss below last checkpoint	the run is stuck
not collapsing	is effective dim ≥ 85% of best seen	the rank-1 shortcut
regularised	is SIGReg within 5× its own null	latent drifting non-Gaussian
representing	is the height probe rising	the one a collapsed run cannot fake

Below a floor, and not improving across two consecutive checkpoints, the run raises `GateAbort` and stops — a dead run costs 35 minutes instead of the full budget. The requirement for a *trend* rather than a single reading is itself measured: a healthy run scored 0.12 at 10%, and aborting on one low number would have killed it.

13 · Six bugs, and how each surfaced

Recorded because the pattern is the point: none of these were visible from reading the code.

BUG	HOW IT SURFACED	WHAT IT WOULD HAVE COST
Gate accepted a collapsing run	reading effective dim next to the loss	a "0.939 gain" headline on a rank-1 model
Diagnostics scored against the wrong sample count	computing the null by hand	healthy runs reported as 6.7× collapsed
Identical data order every epoch	iterating the loader twice and diffing	epochs 2–3 replay epoch 1; shuffling did nothing
SIGReg ran in bf16 under autocast	emulating autocast against fp32	1.0% error — the size of the entire null
LR schedule built on a 14.5%-low estimate	counting real windows vs the formula	last ~15% of every run at ~zero learning rate
Stale BatchNorm statistics at evaluation	taking a mid-training checkpoint and re-measuring	y-probe −0.028 on an encoder measuring +0.121

The last one is the most instructive. Every downstream measurement runs the encoder in `eval()`, where BatchNorm uses running statistics that lag while the encoder is still learning. End-of-run measurements are unaffected because the learning rate has annealed — which is exactly why it survives undetected until someone takes a mid-training checkpoint.

14 · Built but not yet run

STAGE	STATE
Full-budget training	Configured and launched twice, stopped both times for the audit above. 38,104 steps, ~1.8 h/run.
Sparse autoencoder + λ feature diff	Written and verified end-to-end at <code>d_model=192</code> on synthetic latents. Reports which features each λ destroys and cross-references them against RAM ground truth, so the answer is "the features that vanish are the ones tracking height" rather than "37 features changed".
Planner (categorical CEM over 5 macro-actions, probe-scored cost)	Written, 11 unit tests, never run against a trained model. Categorical rather than Gaussian-over-buttons on purpose; sub-goals capped at 200 px, inside the certified range from §10.
SAE-feature steering	Written and tested (reversible forward hook on the predictor's residual stream). Not demonstrated.

15 · Limitations

- **Training budget.** Every number here comes from 4,000 steps, ~4% of the planned schedule, on half the dataset. The SIGReg-only result in §11 is therefore also a statement about a young encoder: the honest reading of -0.383 is "did not acquire height at this budget", not "cannot".
- **The aux comparison is not a representation-quality result** (§11 box).
- **One environment, one level, one visual style.** §4–§7 are properties of the statistic and should transfer; §8–§11 are empirical and may not.
- **The paper's claims are paraphrased here, not quoted.** The measurements are ours; the characterisation of LeJEPa's position in §8 should be checked against the paper text before publication.
- **Probe capacity is a confound in absolute R^2 .** The MLP probes are held fixed across all comparisons, so the comparisons stand, but the absolute numbers depend on that choice.

16 · Practice, and where everything lives

1. Report `SIGReg / null(n)`, never raw SIGReg; publish the batch size next to λ .
2. In video or RL, compute the statistic per frame position, not over the flattened batch.
3. Treat batch size as a SIGReg hyperparameter. Its collapse-detection power is roughly linear in n .
4. Do not assume the stop-gradient is removable outside image SSL at practical batch sizes.
5. Re-estimate BatchNorm statistics before any mid-training evaluation.
6. **Anti-collapse is not representation quality.** Gate on a probe of the state the downstream task actually needs — it costs minutes, and it is the only measurement here that a collapsed-but-confident model cannot fake.

ARTEFACT	LOCATION
code, 161 automated checks	<code>~/aq-mario</code>
8,000 episodes, 27.4 GB	Modal volume <code>aqmario-data</code>
checkpoints, gates, PPO explorer	Modal volume <code>aqmario-runs</code>
live stage board	<code>python -m aqmario.tracker</code>

the settings, with why each was chosen

`recipe.yaml` — pinned to the code by test
